

Poster Presenter Guide — GPU Monte Carlo Option Pricing (poster_v4)

The 60-second pitch

"Monte Carlo is the standard way to price options, but its error shrinks only as $1/\sqrt{N}$ — so you need tens of millions of simulated paths, and a CPU takes seconds per price. Each path is independent, which makes this a textbook GPU problem. We built three CUDA kernels, each isolating one performance lesson. The **naive** kernel gives every path its own thread and atomicAdd — the profiler shows the SMs doing useful work only 2.8% of the time, because millions of atomics serialize in L2. The **shared** kernel fixes that with a warp-shuffle + shared-memory reduction: one atomic per block instead of two per path, and SM throughput jumps to 79%. The **antithetic** kernel then halves the variance for free by mirroring every random draw. The change that mattered most, though, was the **grid-stride refactor**: a fixed 16K-thread grid striding over paths makes RNG state $O(\text{threads})$ instead of $O(N)$ — init gets $65\times$ cheaper and the kernels flip from memory-bound to compute-bound. End result: **$382\times$ end-to-end over a single-core CPU at 10 million paths, $980\times$ at 50 million**, with every run validated against the closed-form Black–Scholes price."

Walking order (matches the numbered panels)

Overview chips (the strip is your map) → **1 naive** → **2 shared** → **3 antithetic** → **4 grid-stride** → **5 roofline** → results charts → hero band numbers.

One sentence per panel if the visitor is in a hurry:

1. **Naive** — "94% occupancy but 2.8% compute: occupancy isn't utilization."
2. **Shared** — "Atomics go from $\sim 2\cdot N$ to ~ 128 ; the diagram below is the whole trick."
3. **Antithetic** — "Pair +Z with -Z, noise cancels: half the variance, one extra $\exp()$."
4. **Grid-stride** — "The table is before/after: RNG init 44.6 ms → 0.68 ms. Biggest single win."
5. **Roofline** — "Proof, not vibes: optimized kernels sit on the FP32 compute ceiling; naive is stuck far below."

Anticipated questions (and honest answers)

- **"Why is the naive curve rising in the validation plot?"** At 5×10^7 paths the naive kernel's FP32 atomicAdd accumulation degrades — adding tiny payoffs into one huge float sum loses precision. The hierarchical reduction avoids this. It's a correctness argument for the optimized kernels, not just a speed one.

- **"1.4 σ for naive/shared — is that a failure?"** No. σ here is the Monte Carlo standard error; $\leq 1.4\sigma$ is statistically consistent. Antithetic lands at 0.8σ . Our automated gate accepts runs within 4σ .
- **"Why a GTX 1650 and not the Colab T4/A100?"** Development happened on Colab, but benchmarks were run locally for stable clocks and exclusive access. Same sm_75 architecture as the T4.
- **"Why FP32?"** Consumer GPUs have $\sim 1/32$ FP64 throughput; FP32 with a validated error budget is the right engineering trade. The precision analysis (naive blow-up) is exactly why we measure it.
- **"Is 382 $\times$ a fair comparison?"** It's end-to-end (RNG init + kernel) vs a single CPU thread, median of 5 runs. Kernel-only is $\approx 1652\times$; a multi-threaded CPU would close some gap — we say so if asked.
- **"Why one thread = one path?"** European options need only the terminal price — one exact GBM step. For path-dependent (Asian) options we'd revisit the mapping.

Key numbers (memorize these five)

Number

382 $\times$ / 980 $\times$

$\approx 1652\times$

2.8% $\rightarrow$ 79%

65 $\times$

0.8σ

These five are the whole story: two say *how fast* (382 $\times$ /980 $\times$, 1652 $\times$), two say *why* (2.8 $\rightarrow$ 79%, 65 $\times$), one says *it's correct* (0.8σ).

How to explain each number

382 $\times$ / 980 $\times$ (end-to-end). Same job — simulate N paths and produce a price — timed wall-to-wall on both sides. One CPU core does ~ 30 M paths/s $\rightarrow \approx 360$ ms at 10^7 ; the GPU (mc_antithetic, RNG init + kernel + readback included) ≈ 0.9 ms $\rightarrow$ 382 $\times$. At 5×10^7 : ≈ 1.6 s vs ≈ 1.6 ms $\rightarrow$ 980 $\times$. Two numbers because speedup grows with N as GPU fixed costs amortize — quoting both is the honest version.

$\approx 1652\times$ (kernel-only). Same comparison but simulation kernel time only (init excluded). Read it together with 980 $\times$: "one price from cold start = 980 $\times$; many prices back-to-back, init paid once $\rightarrow$ approaches 1652 $\times$."

2.8% $\rightarrow$ 79% (SM throughput). Nsight's measure of how much of peak compute the SMs actually sustained. Naive: every path atomicAdds the same two global addresses $\rightarrow$ atomics serialize in L2 $\rightarrow$ cores idle $\sim 97\% \rightarrow$ 2.8%, despite 94% occupancy (resident $\neq$ working). Shared: register $\rightarrow$ warp shuffle $\rightarrow$ shared-memory tree $\rightarrow$ one atomic per block (~ 128 total) $\rightarrow$ 79%. Same math, only the summation changed — 28 $\times$ better utilization. This is the poster's core lesson.

65× (RNG init). One-thread-per-path needs N cuRAND states ($48 \text{ B} \times 10^6 \approx 46 \text{ MB}$); `init_rng` alone took 44.6 ms at $N=10^6$ — longer than the kernel itself. The fixed 16,384-thread grid-stride needs only 16,384 states $\rightarrow 0.68 \text{ ms} = 65\times$. The side effect matters most: state traffic gone, kernels flip memory-bound $\rightarrow$ compute-bound (exactly what the roofline panel shows).

0.8 σ (correctness). σ = Monte Carlo standard-error units. Antithetic at $N=10^6$: estimate 10.4593 vs analytical 10.4506 $\rightarrow$ error 0.0088; its $\text{SE} \approx 0.011 \rightarrow 0.8\sigma$ — the deviation is pure sampling noise, no bug. Values near 1σ are *healthy* for an unbiased estimator; don't confuse this with the 4σ acceptance gate.

Use them in this order: "We're 382× faster end-to-end — 1652× kernel-only — because we fixed the atomic bottleneck (2.8% $\rightarrow$ 79%) and made RNG init 65× cheaper; and the answer is right: 0.8 σ from the analytical price."

Demo line (if the live demo is running)

"Pick any N — price and runtime update instantly; the analytical value is printed next to it, so you can watch the error shrink as $1/\sqrt{N}$."